Pinacoteca
Project Documentation

Elettro5.B

March 15, 2026

Abstract

This documentation describes the Pinacoteca project in full, with a focus on software architecture, control logic, simulation, and validation. It includes requirements, traceability, design rationale, test procedures, and known limits, providing a professional and verifiable overview.

Contents

Abstract **i**

1 Introduction **1**

2 Architecture **2**

2.1 Logical layout 2

2.2 Data flow 2

2.3 Timing cycles 2

2.4 Key parameters 2

3 Design decisions and safety **3**

3.1 Modularity and responsibility 3

3.2 Sensor error handling 3

3.3 Actuator protection and stability 3

3.4 Interface and communication choices 3

3.5 Testing strategy 3

4 Diagrams **4**

4.1 Block view 4

4.2 Loop logical sequence 4

5 Hardware **5**

5.1 Components 5

5.2 Selection criteria 5

5.3 Pin map 5

5.4 Graphical wiring schematic 6

5.5 Wiring notes 6

6 Firmware **7**

6.1 Setup 7

6.2 Loop 7

6.3 Error handling 7

6.4 Target configuration 7

7 Software modules **9**

7.1 LED . . . 9
7.2 Servomotor and turnstile . . . 9
7.3 Stoplight . . . 9
7.4 Temperature and thermostat . . . 10
7.5 Humidity and control . . . 10
7.6 Lighting . . . 10

8 LCD Display **12**
8.1 Pages . . . 12
8.2 Date and time computation . . . 12
8.3 Update . . . 12
8.4 Safety and robustness . . . 12

9 Step-by-step implementation **13**
9.1 Global initialization . . . 13
9.2 Turnstile and people counting . . . 13
9.3 Capacity stoplight . . . 13
9.4 Thermostat . . . 14
9.5 Humidity control . . . 14
9.6 Lighting . . . 14
9.7 LCD display . . . 15

10 Requirements and traceability **16**
10.1 Functional requirements . . . 16
10.2 Test traceability . . . 16
10.3 Notes . . . 16

11 Tests **17**
11.1 Structure . . . 17
11.2 Execution . . . 17
11.3 Test rationale . . . 17
11.3.1 LED . . . 17
11.3.2 Stoplight . . . 17
11.3.3 Servomotor . . . 18
11.3.4 Turnstile . . . 18
11.3.5 Temperature . . . 18

11.3.6 Thermostat . . . 18
11.3.7 Humidity and HumidifierControl . . . 18
11.3.8 Photoresistor and LightingControl . . . 19
11.3.9 DisplayInterface . . . 19
11.3.10 DisplayPanel . . . 19
11.4 run_tests.sh script . . . 19

12 Usage instructions **20**
12.1 Test build . . . 20
12.2 Arduino compilation . . . 20
12.3 Wokwi simulation . . . 20

13 Risks and failure modes **21**
13.1 Sensor risks . . . 21
13.2 Actuator risks . . . 21
13.3 Integration risks . . . 21

14 Timing and performance **22**
14.1 Update frequencies . . . 22
14.2 Motivations . . . 22
14.3 Impact on the loop . . . 22

15 Known limits and roadmap **23**
15.1 Current limits . . . 23
15.2 Roadmap . . . 23

16 Requirements-design-test traceability **24**
16.1 Traceability matrix . . . 24
16.2 Notes . . . 24

17 Software validation and simulation **25**
17.1 Validation strategy . . . 25
17.2 Simulation validation procedures . . . 25
17.3 Software acceptance criteria . . . 25

18 Calibration and simulation limits **26**
18.1 Software calibration . . . 26
18.2 Simulation limits . . . 26

19 Trade-offs and design choices **27**
19.1 Time management . . . 27
19.2 Sentinel values for errors . . . 27
19.3 Modular architecture . . . 27

20 Glossary **28**

21 Appendix: public APIs **29**
21.1 Turnstile . . . 29
21.2 Stoplight . . . 29
21.3 Thermostat . . . 29
21.4 HumidifierControl . . . 29
21.5 LightingControl . . . 30
21.6 DisplayPanel . . . 30
21.7 Utility functions . . . 30

22 Software and dependencies **31**
22.1 Tools . . . 31
22.2 Libraries . . . 31
22.3 Repository . . . 31

23 Remote Control, App Communication, and Stability Improvements **32**
23.1 Purpose of this chapter . . . 32
23.2 Remote communication architecture . . . 32
23.3 Protocol used by the app . . . 32
23.3.1 Core commands . . . 32
23.3.2 Setpoint commands . . . 32
23.3.3 Manual override commands . . . 33
23.3.4 State payload format . . . 33
23.3.5 Error model . . . 33
23.4 Validation and test strategy . . . 33
23.5 Software optimization for stability . . . 33
23.6 Wokwi issues encountered and fixes . . . 34
23.6.1 Issue A: board and firmware mismatch . . . 34
23.6.2 Issue B: duplicated key in `wokwi.toml` . . . 34
23.6.3 Issue C: servo not visibly moving in simulation . . . 34

23.7 Build workflow (single command) 34
23.8 No-hardware app testing 34
23.9 Final deployment note 35

Chapter 1— Introduction

The **Pinacoteca** project delivers a management system for a gallery, covering access control, climate, humidity, lighting, and LCD visualization. The goal is to integrate sensors and actuators into a modular Arduino firmware, with unit tests runnable on a PC and a repeatable build flow.

This documentation is meant to describe both the technical solution and the rationale behind design choices. Each section dives into the behavior step by step, highlighting safety aspects, adopted trade-offs, and the reasoning behind the exposed APIs.

The main features are:

- counting entrances and exits with a servo-assisted turnstile;
- indicating available capacity with a LED traffic light;
- temperature regulation with heating or cooling modes;
- humidity control with threshold and actuator (LED in simulation);
- ambient light control with a photoresistor and dimmer;
- a 16x2 LCD panel with informative pages.

The code is structured into independent modules under `lib/`. This choice reduces cross-dependencies, makes behavior easier to read, and simplifies validation through unit tests.

The system is built around reliability: if a sensor reading is invalid, control modules switch to a fallback behavior that turns outputs off, avoiding unsafe or non-deterministic states.

Chapter 2— Architecture

The firmware follows a deterministic execution cycle: initialization in `setup()` and periodic module updates in `loop()`. Each module exposes `begin()` and `update()` to separate configuration from runtime logic, reducing the risk of inconsistent states.

2.1 Logical layout

- **Turnstile**: reads IN/OUT buttons, updates the people count, and moves the servo.
- **Stoplight**: displays availability using green/red LEDs.
- **Thermostat**: reads the NTC sensor and decides heating/cooling.
- **LightingControl**: reads the photoresistor and adjusts the dimmer.
- **HumidifierControl**: reads the DHT22 and turns humidity control on or off.
- **DisplayPanel**: alternates LCD pages with status, date/time, and diagnostics.

2.2 Data flow

The flow follows three constant steps:

1. **Acquisition**: analog or digital readings from sensors.
2. **Validation**: error checks and threshold filters.
3. **Actuation**: LED, PWM, or servo based on decisions.

The display aggregates module states and presents a concise view for the operator.

2.3 Timing cycles

Some modules use internal timing to avoid rapid transitions or flicker. In particular:

- the thermostat applies a pause before switching mode;
- the display caps refresh at 1 second and switches page every 5 seconds.

These choices prevent oscillations and improve perceived stability.

2.4 Key parameters

Target values are configured in the main file:
`20.0 C` for temperature, `65%` for humidity, `200 lux` for lighting, `5 people` as the maximum capacity.

Chapter 3— Design decisions and safety

This chapter explains the technical choices and the implemented safety measures. The goal is a robust, predictable, and easily testable firmware.

3.1 Modularity and responsibility

Splitting the code into modules under `lib/` addresses three needs: isolate sensor logic, simplify reuse, and enable focused unit tests. Each module has a single responsibility and a minimal API surface.

The separation between `begin()` and `update()` distinguishes hardware configuration from runtime behavior. This reduces side effects and makes logic observable even in tests using mocks.

3.2 Sensor error handling

For analog and digital sensors, a consistent sentinel value is used: `-999.0` for read errors. This choice makes it easy to identify errors downstream without exceptions or complex state.

Each control module validates readings before actuating. If input is invalid, the module turns outputs off (fail-safe) and returns `false`. This prevents silent error propagation.

3.3 Actuator protection and stability

In the servo module, `antiSufferingServo` prevents motion beyond mechanical limits, logging a message on Serial. This avoids stalls or motor stress.

The thermostat introduces a pause, defined by `_pauseTime`, to avoid rapid toggling when readings oscillate near the threshold. This improves reliability and actuator lifespan.

Lighting control uses a PWM range limited to 0–255. In Wokwi simulation, the value is forced to 255 when light is needed, ensuring visual consistency and reducing ambiguity in tests.

3.4 Interface and communication choices

The LCD is updated at most once per second and rotates pages every 5 seconds. This policy protects against excessive writes, avoids flicker, and preserves readability.

The error page summarizes operational issues (humidity high/low, abnormal temperature, people count out of range) with short texts. The choice is oriented toward quick diagnostics in real environments.

3.5 Testing strategy

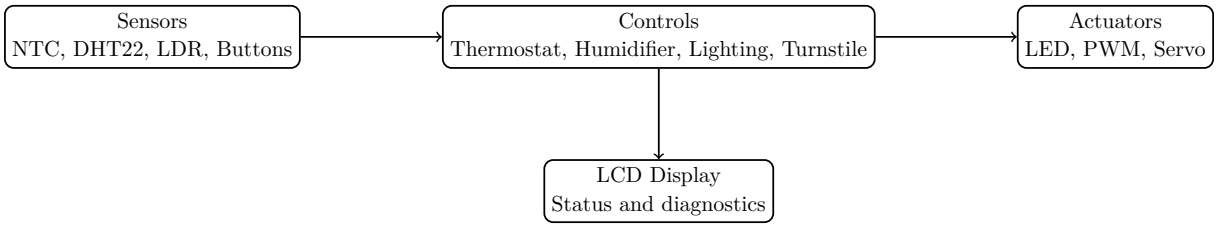
The test suite validates edge conditions: sensor limits, actuator limits, state transitions, and safety fallback. Each test checks both logical output and side effects on mocked I/O.

Local Arduino mocks allow tests to run on a PC, speeding up feedback and reducing hardware dependence. This makes the project more maintainable and suitable for future evolution.

Chapter 4— Diagrams

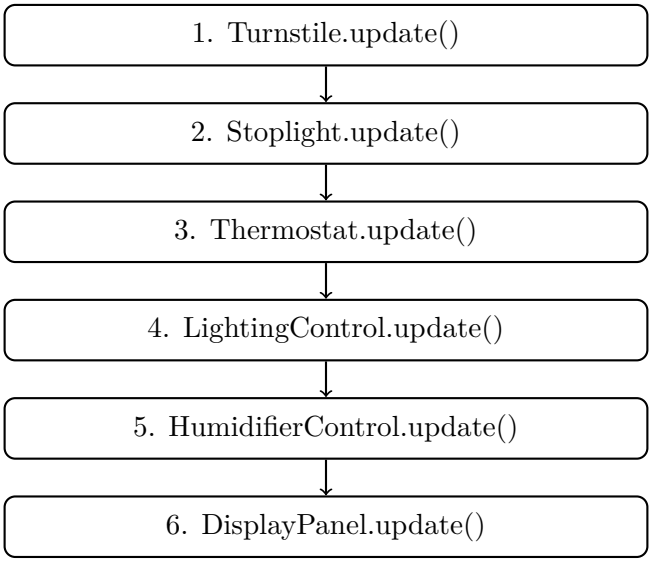
4.1 Block view

The following figure summarizes the main blocks and interactions between sensors, controls, and actuators.



4.2 Loop logical sequence

The update cycle is designed so the display always shows the most recently computed state.



Chapter 5— Hardware

The Wokwi simulation described in `diagram.json` uses an Arduino Uno, a servo, buttons, LEDs, sensors, and a 16x2 LCD. The wiring matches the pins defined in the firmware, so the simulation faithfully represents the behavior expected on real hardware.

5.1 Components

- Arduino Uno
- Servo for the turnstile
- 2 buttons (IN/OUT) with pull-down resistors
- Green and red LEDs (traffic light)
- Blue and orange LEDs (cooling/heating)
- White LED (humidity)
- Photoresistor (LDR)
- NTC sensor for temperature
- DHT22 sensor for humidity
- 4 yellow LEDs for the ceiling light (driven in parallel)
- 16x2 LCD

5.2 Selection criteria

Components were chosen for easy integration and the availability of stable libraries. The NTC provides a linear analog measure for the thermostat, while the DHT22 provides humidity with a simple digital interface. The 16x2 LCD balances cost and readability, with a standardized interface via `LiquidCrystal`.

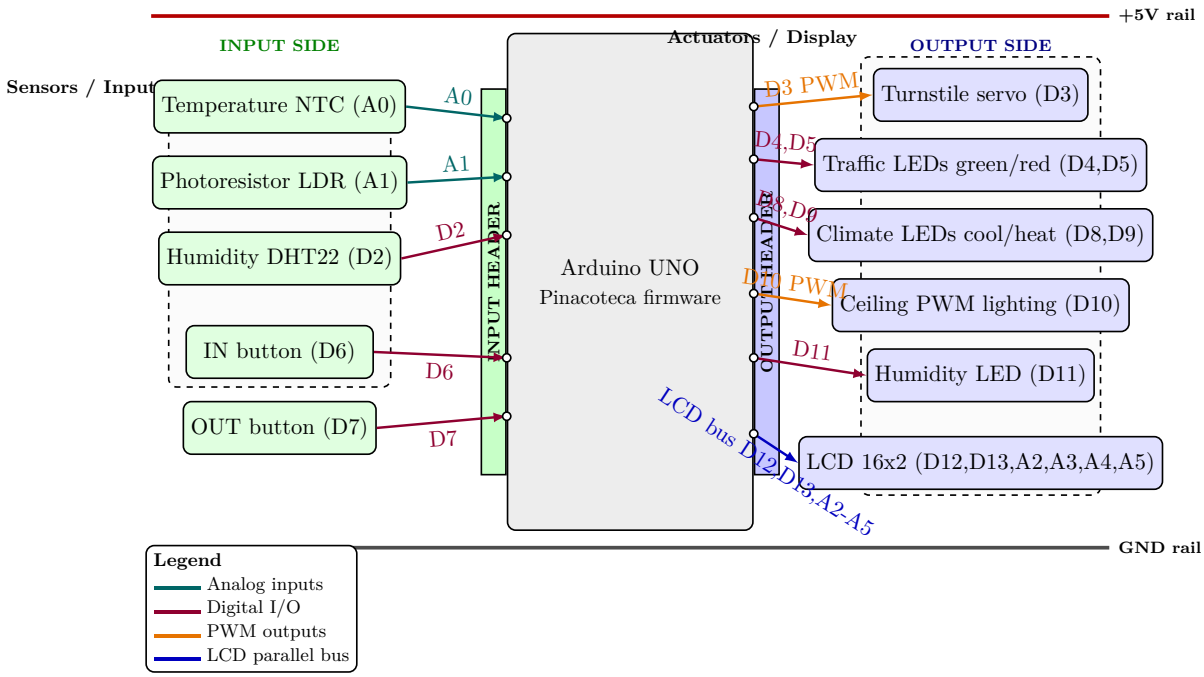
5.3 Pin map

Function	Pin	Notes
Turnstile servo	D3	PWM
IN button	D6	digital input
OUT button	D7	digital input
Traffic light green LED	D4	digital output
Traffic light red LED	D5	digital output
Cooling LED	D8	digital output
Heating LED	D9	digital output
Ceiling light (PWM)	D10	lighting dimmer
Humidity LED	D11	digital output
Temperature sensor (NTC)	A0	analog input
Photoresistor	A1	analog input
LCD RS	D12	
LCD EN	D13	
LCD D4	A2	

LCD D5	A3	
LCD D6	A4	
LCD D7	A5	
DHT22	D2	pin defined in `humidity.h`

5.4 Graphical wiring schematic

The following figure provides a compact wiring overview with Arduino at the center, sensors on the left, and actuators/display on the right. Connections are grouped by signal type (analog, digital, PWM, LCD bus) to improve readability while preserving firmware pin mapping.



5.5 Wiring notes

- LEDs are connected with 220 Ωresistors to limit current.
- Buttons use 10 kΩpull-down resistors to avoid floating readings.
- The DHT22 uses digital pin 2 and requires 5V power.
- The ceiling light is driven in PWM on D10, which simplifies intensity control.

Chapter 6— Firmware

The main file `Pinacoteca.ino` instantiates modules and defines pins. The logic is organized in two distinct phases, providing a predictable and easily verifiable execution cycle.

6.1 Setup

During `setup()` the following are initialized:

- serial for debug and diagnostics;
- servo and turnstile with a known initial position;
- traffic light, thermostat, lighting, and humidity;
- LCD panel with a startup message.

Initializing the servo before other modules reduces the risk of unintended movement during pin configuration.

6.2 Loop

In the main loop, each module updates its state in a coherent sequence:

1. update turnstile and people count;
2. update traffic light based on the limit;
3. adjust climate (heating or cooling);
4. adjust lighting;
5. control humidity;
6. refresh the display with current data.

The order is intentional: human inputs are handled first, then status indicators are updated, and finally the LCD interface is refreshed. This way the display reflects the latest decision of the cycle.

6.3 Error handling

Modules that read sensors validate measurement correctness. If input is invalid, outputs are set to a safe state and the update function returns `false`. This behavior aligns with the project fail-safe philosophy.

6.4 Target configuration

Targets are passed to module constructors:

- `Thermostat`: desired temperature in Celsius.
- `LightingControl`: target illuminance in lux.
- `HumidifierControl`: target humidity in percent.
- `Turnstile`: maximum number of people.

Centralized configuration in the main file allows quick tuning without modifying individual modules.

Chapter 7— Software modules

This section describes in detail the modules in `lib/`. Each module is designed with a minimal interface and explicit internal logic, easing reuse and verification through tests.

7.1 LED

Purpose The `led.h` file provides actuation primitives to write to digital or PWM pins. Centralizing these functions avoids duplication across control modules.

Flow `led(pin, state)` performs a single `digitalWrite` and returns `true`. `ledDimming(pin, pwm)` performs `analogWrite` with the same scheme. A uniform return semantics simplifies tests.

Safety The functions do not implement safety logic, but are used in a context where calling modules validate their inputs. This prevents a sensor error from being turned into an active actuator.

Choices Keeping functions simple and stateless reduces complexity and improves behavioral predictability in tests with mocks.

7.2 Servomotor and turnstile

Purpose The `servomotor.h` module protects the servo via `antiSufferingServo`. The `turnstile.h` module manages entrances and exits, maintaining a people counter.

Flow In `Turnstile::begin`, button pins are configured and the servo is attached. In `update()`, IN/OUT buttons are read, capacity constraints are applied, and the servo is moved for opening and closing.

Safety `antiSufferingServo` prevents exceeding the 0–180 range, logs a message on Serial, and returns `false`. This reduces the risk of mechanical damage. The counter prevents entries above the maximum or exits below zero.

Choices An internal counter avoids external dependencies and makes the module self-contained. Checking limits before moving the servo is a safety choice that avoids unnecessary actuation.

7.3 Stoplight

Purpose `stoplight.h` translates capacity state into immediate visual signals.

Flow `begin()` configures pins as outputs. `update(currentPeople, maxPeople)` decides whether to turn on green or red based on the comparison.

Safety No intermediate modes are provided: in unexpected conditions, the module always keeps one of the two outputs active, avoiding ambiguity for the operator.

Choices Binary logic makes behavior easy to verify and minimizes interpretation errors.

7.4 Temperature and thermostat

Purpose `temperature.h` converts the NTC analog reading into Celsius. `thermostat.h` decides the climate mode based on a central threshold with tolerance.

Flow The temperature reader checks extreme values and returns `-999.0` on error. The thermostat reads the temperature, determines the requested action, and activates heating or cooling.

Safety If the reading is invalid, the thermostat turns both outputs off and returns `false`. In addition, a time pause between state changes prevents rapid toggling that could wear components.

Choices The 1.0 C tolerance creates a neutral window that stabilizes the system. The pause is a compromise between responsiveness and stability, suitable for a real environment.

7.5 Humidity and control

Purpose `humidity.h` encapsulates DHT22 usage and provides a humidity reading. `humidifier.h` manages the actuator based on the target.

Flow The sensor is initialized with `beginHumiditySensor`. `readHumidity()` returns the current value or `-999.0` on error. The humidity control compares the reading with the target and applies a tolerance.

Safety On sensor error, the output is switched off. The tolerance avoids continuous oscillations around the setpoint.

Choices A single sentinel value for errors simplifies integration with the display and diagnostics.

7.6 Lighting

Purpose `photoresistor.h` estimates lux from a photoresistor. `lighting_control.h` regulates the ceiling light via PWM.

Flow The analog reading is converted to lux using a simplified model. The control calculates the difference from the target and decides whether to activate PWM.

Safety Out-of-range readings are handled with minimum or maximum values, avoiding divisions by zero. PWM is always limited to the 0–255 range.

Choices In Wokwi simulation the PWM is set to 255 when light is needed, providing clear and predictable visual behavior during tests.

Chapter 8— LCD Display

The 16x2 display uses an abstraction layer defined in `display.h` (class `DisplayInterface`) and application logic in `lcd.h` (class `DisplayPanel`). This separation allows logic to be tested without directly depending on hardware.

8.1 Pages

The panel rotates three pages every 5 seconds:

- **Date/Time**: calculated from `__DATE__`, `__TIME__`, and `millis()`.
- **Status**: temperature, humidity, and people count.
- **Diagnostics**: error messages or anomalies.

The error page shows critical conditions with priority, enabling quick intervention.

8.2 Date and time computation

Since there is no RTC, time is computed by combining the build timestamp with the increment of `millis()`. The base is set in the constructor, offering a coherent initial date and continuous time progression.

8.3 Update

Refresh is limited to 1 second to avoid flicker. Each update reads values from `Thermostat`, `HumidifierControl`, and `LightingControl`. The page switches every 5 seconds to allow time to read each set of information.

8.4 Safety and robustness

If a sensor value is invalid, the display shows `ERR` or an explicit message. This avoids misinterpretation and makes faults evident.

Chapter 9— Step-by-step implementation

This chapter describes the operational flow for each feature, with code fragments and detailed explanations.

9.1 Global initialization

In the main file, modules are instantiated with key parameters and centrally defined pins. This choice makes the configuration easy to read.

```
const int PIN_SERVOMOTOR = 3;
const int PIN_GREEN_LIGHT = 4;
const int PIN_RED_LIGHT = 5;

Turnstile entranceTurnstile(PIN_IN_BUTTON, PIN_OUT_BUTTON, 5);
Stoplight trafficLight(PIN_GREEN_LIGHT, PIN_RED_LIGHT);
Thermostat mainThermostat(PIN_TEMPERATURE_SENSOR, 20.0, PIN_YELLOW_LIGHT,
    PIN_BLU_LIGHT);
LightingControl galleryLighting(PIN_PHOTORESISTOR, PIN_PLAFONIERE, 200);
HumidifierControl galleryHumidifier(IS_HUMIDIFIER_LED, 65.0);
DisplayPanel galleryDisplay(PIN_LCD_RS, PIN_LCD_EN, PIN_LCD_D4, PIN_LCD_D5, PIN_LCD_D6,
     PIN_LCD_D7, 5);
```

Rationale Targets and capacity are centralized to reduce errors and enable quick calibration.

9.2 Turnstile and people counting

The turnstile reads buttons and moves the servo with mechanical protection.

```
if (inState == HIGH && _currentPeople < _maxPeople) {
    moveServo(90);
    _currentPeople++;
    delay(1000);
    moveServo(-90);
}
```

Steps 1) read inputs, 2) check capacity, 3) open servo, 4) update counter, 5) close servo.

Safety The counter prevents entries beyond the limit and `antiSufferingServo` prevents out-of-range movement.

9.3 Capacity stoplight

The stoplight translates the count into an immediately readable state.

```
if (currentPeople < maxPeople) {
    led(_greenPin, HIGH);
    led(_redPin, LOW);
} else {
```

```
    led(_greenPin, LOW);
    led(_redPin, HIGH);
}
```

Rationale Two states reduce ambiguity for the operator.

9.4 Thermostat

The thermostat reads temperature and decides the action with tolerance and an anti-toggle pause.

```
if (temperature_celsius < (_desiredTemperature - tolerance)) {
    desired_action = 1;
} else if (temperature_celsius > (_desiredTemperature + tolerance)) {
    desired_action = 2;
} else {
    desired_action = 0;
}
```

Steps 1) read sensor, 2) validate, 3) compare with target, 4) choose action, 5) actuate with pause.

Safety On invalid reading, both outputs are turned off and the method returns `false`.

9.5 Humidity control

The module compares the DHT22 reading with the target and a fixed tolerance.

```
if (currentHumidity > (_targetHumidity + tolerance)) {
    led(_humidifierPin, HIGH);
} else if (currentHumidity < (_targetHumidity - tolerance)) {
    led(_humidifierPin, LOW);
} else {
    led(_humidifierPin, LOW);
}
```

Safety If the sensor returns `NAN`, the reading becomes `-999.0` and the module turns the output off.

9.6 Lighting

Lighting control uses a simple model to translate the analog reading into lux and decide PWM.

```
float missingLight = _targetLux - currentLux;
if (missingLight > 0) {
    pwm_value = 255;
} else {
    pwm_value = 0;
}
ledDimming(_dimmerPin, pwm_value);
```

Rationale In Wokwi simulation a full PWM is used when light is needed, making the response evident.

9.7 LCD display

The display rotates pages with different information and shows errors with priority.

```
if (now - _lastPageChange >= 5000) {
    _page = (_page + 1) % 3;
    _lastPageChange = now;
}
```

Steps 1) limit refresh, 2) rotate page, 3) read values from modules, 4) print selected page.

Safety Invalid readings are shown as `ERR` or error messages, preventing misinterpretation.

Chapter 10— Requirements and traceability

This section lists the main functional requirements and their traceability to unit tests. The goal is to demonstrate that design choices are verified systematically.

10.1 Functional requirements

- **R1** The system must count entrances and exits with a configurable maximum limit.
- **R2** The system must signal capacity status with a green/red traffic light.
- **R3** The system must control temperature with heating and cooling modes.
- **R4** The system must control humidity with threshold and tolerance.
- **R5** The system must control lighting with a target in lux.
- **R6** The display must show date/time, status, and diagnostics.
- **R7** In case of sensor errors, actuators must be deactivated.

10.2 Test traceability

Req.	Description	Related tests
R1	People counting with limits	`TurnstileIncrementsAndDecrementsPeople` `TurnstileRespectsMaxPeople`
R2	Capacity stoplight	`StoplightBelowMaxSetsGreen` `StoplightAtMaxSetsRed`
R3	Climate with heating/cooling	`ThermostatHeatingBranch` `ThermostatCoolingBranch`
R4	Humidity control	`HumidifierHighHumidityTurnsOnPin` `HumidifierLowHumidityTurnsOffPin`
R5	Lighting control	`LightingControlWritesPwm`
R6	Display pages	`DisplayPanelRotatesPages` `DisplayDatePageHasValidWeekdayPrefix`
R7	Fail-safe on sensor errors	`ThermostatSensorErrorTurnsEverythingOff` `HumidifierSensorErrorReturnsFalse`

10.3 Notes

Tests are executed with Arduino mocks, so checks focus on side effects (pins and LCD buffer) rather than real hardware. This approach ensures speed and repeatability.

Chapter 11— Tests

The test suite uses GoogleTest and is configured in the `test/` folder. The goal is to ensure each module reacts correctly to nominal cases and edge cases, with particular attention to safety conditions.

11.1 Structure

The `CMakeLists.txt` file in `test/` downloads GoogleTest via FetchContent. The generated executable is called `pinacoteca_tests` and Arduino API mocks are in `test/mocks/`. The test entry point is `all_tests.cpp`, where each feature group has dedicated tests.

11.2 Execution

Two modes are available:

- Direct CMake usage:

```
cd test
cmake -S . -B build
cmake --build build -j
ctest --test-dir build --output-on-failure
```

- `run_tests.sh` script that manages build, clean, and Arduino compilation.

11.3 Test rationale

Below is the detailed explanation of the checks, with objectives, conditions, and expectations.

11.3.1 LED

- **Objective**: ensure output primitives write to the correct pins.
- **Conditions**: direct calls to `led` and `ledDimming` with known values.
- **Expectations**: `digitalWrite` and `analogWrite` receive the requested value.
- **Tests**: `LedWriteDigitalState`
  `LedDimmingWritesPwm`

11.3.2 Stoplight

- **Objective**: validate green/red semantics based on capacity.
- **Conditions**: counts below and equal to the maximum.
- **Expectations**: green on below capacity, red on at full capacity.
- **Tests**: `StoplightBelowMaxSetsGreen`
  `StoplightAtMaxSetsRed`

11.3.3 Servomotor

- **Objective**: prevent movement beyond mechanical limits.
- **Conditions**: commands with valid angles and angles outside 0–180.
- **Expectations**: movement allowed within range, blocked and logged outside range.
- **Tests**: `AntiSufferingServoAllowsSafeMove`
  `AntiSufferingServoBlocksOutOfRange`

11.3.4 Turnstile

- **Objective**: count entrances/exits correctly and respect the maximum.
- **Conditions**: simulated IN/OUT button states; variable max people.
- **Expectations**: coherent increment and decrement, no limit overruns.
- **Tests**: `TurnstileIncrementsAndDecrementsPeople`
  `TurnstileRespectsMaxPeople`
  `TurnstileBeginConfiguresPins`

11.3.5 Temperature

- **Objective**: handle abnormal NTC sensor readings.
- **Conditions**: analog values at extremes (0 and 1023) and an intermediate value.
- **Expectations**: sentinel `-999.0` at extremes, finite value under normal conditions.
- **Tests**: `TemperatureReaderReturnsErrorAtExtremes`
  `TemperatureReaderReturnsFiniteForValidInput`

11.3.6 Thermostat

- **Objective**: verify heating/cooling logic and fail-safe behavior.
- **Conditions**: valid reading above/below target, and invalid reading.
- **Expectations**: correct pin activation, outputs off on sensor error.
- **Tests**: `ThermostatHeatingBranch`
  `ThermostatCoolingBranch`
  `ThermostatSensorErrorTurnsEverythingOff`
  `ThermostatBeginSetsPinModesAndSetGetTarget`

11.3.7 Humidity and HumidifierControl

- **Objective**: validate DHT22 reading and threshold logic.
- **Conditions**: high humidity, low humidity, and invalid reading (`NAN`).
- **Expectations**: actuator consistent with threshold and off on error.

- **Tests**: `HumidityReadAndError`
  `HumidifierHighHumidityTurnsOnPin`
  `HumidifierLowHumidityTurnsOffPin`
  `HumidifierSensorErrorReturnsFalse`
  `HumidifierSetGetTargetWorks`

11.3.8 Photoresistor and LightingControl

- **Objective**: ensure conversion and PWM align with the target.
- **Conditions**: analog values at extremes and intermediate values.
- **Expectations**: lux clamped at the edges and coherent PWM (255 with low light, 0 with high light).
- **Tests**: `PhotoresistorLuxEdgeCases`
  `LightingControlWritesPwm`
  `LightingControlBeginAndSetGetTarget`

11.3.9 DisplayInterface

- **Objective**: verify the abstract LCD layer.
- **Conditions**: calls to `begin`, `clear`, `setCursor`, `print`.
- **Expectations**: the mock LCD buffer contains the expected strings.
- **Tests**: `DisplayInterfaceBasicOperations`

11.3.10 DisplayPanel

- **Objective**: verify page rotation and diagnostic messages.
- **Conditions**: simulated time advancement and sensor value changes.
- **Expectations**: date/time page, status page, and error page shown in rotation.
- **Tests**: `DisplayPanelRotatesPages`
  `DisplayDatePageHasValidWeekdayPrefix`
  `DisplayStatusPageShowsTurnstileCountExactly`
  `DisplayErrorPageShowsHumidityHigh`
  `DisplayErrorPageShowsTurnstileError`

11.4 run_tests.sh script

Main options:

- `-clean`: deletes the build folder.
- `-with-arduino`: compiles the sketch with arduino-cli.
- `-upload`: uploads the sketch to a board (requires `-port`).
- `-board <fqbn>`: selects the board, default `arduino:avr:uno`.

Chapter 12— Usage instructions

12.1 Test build

To run unit tests:

```
make test
```

This command invokes the script and produces the test executable in the `test/build` folder.

To clean:

```
make test-clean
```

Cleaning is recommended when switching toolchains or updating dependencies.

12.2 Arduino compilation

To compile the sketch via the script:

```
./test/run_tests.sh --with-arduino --board arduino:avr:uno
```

This mode runs unit tests first and then compiles the sketch, reducing the risk of uploading unverified firmware.

To upload to a real board:

```
./test/run_tests.sh --upload --board arduino:avr:uno --port /dev/ttyACM0
```

Before uploading, verify that the serial port is correct.

12.3 Wokwi simulation

The simulation wiring is defined in `diagram.json`. Open the project in Wokwi and start the simulation to observe the overall behavior. The simulation is useful for validating the flow without physical hardware.

Chapter 13— Risks and failure modes

This section identifies the main technical risks and implemented countermeasures. The goal is to demonstrate awareness of failure modes and a realistic mitigation strategy.

13.1 Sensor risks

Risk	Effect	Mitigation
NTC disconnected or shorted	Incorrect or infinite reading	Sentinel value `-999.0`, outputs off
DHT22 not responding	Invalid humidity	Control disabled, error shown on LCD
Photoresistor saturated	Extreme lux	Values clamped (0 or 9999) to avoid division by zero
Noisy buttons	Incorrect counting	Capacity thresholds and increment/decrement logic

13.2 Actuator risks

Risk	Effect	Mitigation
Servo overtravel	Mechanical stress	`antiSufferingServo` blocks beyond 0–180
Rapid climate toggling	Actuator wear	Time pause before switching state
Wrong PWM	Unstable lighting	PWM always in range 0–255

13.3 Integration risks

- **Library dependencies**: mitigated with simple APIs and local mocks for tests.
- **State inconsistency**: mitigated with a fixed sequence in `loop()`.
- **Insufficient diagnostics**: mitigated with an LCD error page and Serial logs.

Chapter 14— Timing and performance

This section describes key firmware timings and the motivations behind temporal choices.

14.1 Update frequencies

- **Display**: refresh every 1 second, page change every 5 seconds.
- **Thermostat**: continuous checks with a 5-second anti-toggle pause.
- **Turnstile**: immediate response to buttons, with a 1-second delay to simulate passage.

14.2 Motivations

- LCD refresh at 1 second avoids flicker and reduces redundant writes.
- The thermostat pause stabilizes behavior near the threshold.
- The turnstile delay avoids rapid double counting.

14.3 Impact on the loop

The module sequence is short and deterministic. The only intentional delay is in the turnstile, chosen to represent physical opening. In a future evolution, this delay can be replaced by a non-blocking `millis()`-based state machine.

Chapter 15— Known limits and roadmap

15.1 Current limits

- PWM lighting control in simulation uses a fixed value due to Wokwi limits.
- The turnstile uses a blocking 1-second delay during opening.
- Time is estimated without an RTC, based on `__DATE__` and `__TIME__`.
- No hardware/software debounce filters are implemented for buttons.

15.2 Roadmap

- Introduce an RTC for accurate date/time.
- Real PWM control with lux feedback (PID or proportional control).
- Non-blocking turnstile control with a state machine.
- Software debounce for buttons and event logic.
- Expand tests with stress and timing cases.

Chapter 16— Requirements-design-test traceability

This section links requirements, software components, and unit tests, focusing exclusively on programming and simulation aspects.

16.1 Traceability matrix

Req.	Software components	Tests
R1	`Turnstile`, `servomotor.h`	`TurnstileIncrementsAndDecrementsPeople`, `TurnstileRespectsMaxPeople`
R2	`Stoplight`, `led.h`	`StoplightBelowMaxSetsGreen`, `StoplightAtMaxSetsRed`
R3	`Thermostat`, `temperature.h`	`ThermostatHeatingBranch`, `ThermostatCoolingBranch`
R4	`HumidifierControl`, `humidity.h`	`HumidityReadAndError`, `HumidifierSensorErrorReturnsFalse`
R5	`LightingControl`, `photoresistor.h`	`PhotoresistorLuxEdgeCases`, `LightingControlWritesPwm`
R6	`DisplayPanel`, `DisplayInterface`	`DisplayPanelRotatesPages`, `DisplayInterfaceBasicOperations`
R7	All control modules	`ThermostatSensorErrorTurnsEverythingOff`, `HumidifierSensorErrorReturnsFalse`

16.2 Notes

Traceability shows that each software requirement is covered by at least one test and that modules are independent and verifiable in isolation via mocks.

Chapter 17— Software validation and simulation

This section describes how to validate firmware behavior in simulation and in a test environment, without entering electrical aspects.

17.1 Validation strategy

- **Unit tests**: cover base functions and module logic.
- **Wokwi simulation**: end-to-end verification of the `loop()` cycle with virtual sensors and actuators.
- **Regression tests**: repeated suite execution after code changes.

17.2 Simulation validation procedures

1. Start the simulation and verify display initialization.
2. Press IN/OUT buttons and verify people count and stoplight.
3. Force high/low temperatures and observe heating/cooling LEDs.
4. Change humidity and observe control activation.
5. Darken or illuminate the photoresistor and verify the ceiling light.
6. Check LCD page rotation and error messages.

17.3 Software acceptance criteria

- No crashes or blocks in the `loop()` cycle.
- All modules respond coherently to simulated input values.
- Error conditions deactivate outputs and show diagnostics.
- The test suite completes successfully.

Chapter 18— Calibration and simulation limits

This section describes software calibration aspects and simulation limits.

18.1 Software calibration

- **Temperature**: the NTC calculation uses standard parameters (BETA, RL10). Real accuracy depends on the sensor and wiring, but the simulation model is stable.
- **Humidity**: the DHT22 in simulation returns ideal values; in a real environment an empirical calibration is recommended.
- **Lux**: conversion from the photoresistor is approximate; the target lux is a software reference rather than a certified measurement.

18.2 Simulation limits

- PWM is simplified (full value when light is needed).
- Sensor response time is idealized.
- Some real phenomena (noise, drift, latency) are not reproduced.

Chapter 19— Trade-offs and design choices

This section summarizes some evaluated alternatives and the motivations for the adopted solution, limited to software aspects.

19.1 Time management

- **Choice**: use `delay` in the turnstile and `millis()` for display/thermostat.
- **Motivation**: simple to understand and sufficient in simulation.
- **Trade-off**: `delay` is blocking; an FSM would be more scalable.

19.2 Sentinel values for errors

- **Choice**: `-999.0` for sensor errors.
- **Motivation**: easy to recognize and propagate.
- **Trade-off**: requires explicit checks in each module.

19.3 Modular architecture

- **Choice**: independent modules with small APIs.
- **Motivation**: facilitates testing and maintenance.
- **Trade-off**: higher number of files and interfaces to maintain.

Chapter 20— Glossary

Term	Meaning
Fail-safe	Behavior that moves the system to a safe state on error.
FSM	Finite state machine used to manage events over time.
Lux	Unit of illuminance.
PWM	Pulse-width modulation used to regulate power.
Sentinel	Special value indicating an error or invalid state.
Wokwi	Online simulator for Arduino and electronic components.

Chapter 21— Appendix: public APIs

This appendix summarizes the main classes and functions exposed by the modules. APIs are designed to be minimal and easy to test, with a common `begin()` + `update()` pattern for control modules.

21.1 Turnstile

Manages people counting and servo actuation based on button inputs.

- `Turnstile(int inButton, int outButton, int maxPeople)`
- `void begin(Servo* servo)`
- `void update()`
- `int getPeopleCount() const`

21.2 Stoplight

Translates capacity into a red/green visual signal.

- `Stoplight(int greenPin, int redPin)`
- `void begin()`
- `void update(int currentPeople, int maxPeople)`

21.3 Thermostat

Controls heating and cooling with tolerance and an anti-toggle pause.

- `Thermostat(int sensorPin, float desiredTemperature, int heatingPin, int coolingPin)`
- `void begin()`
- `bool update()`
- `void setTargetTemperature(float newTarget)`
- `float getTargetTemperature() const`
- `float getCurrentTemperature() const`

21.4 HumidifierControl

Controls humidity with sentinel handling in case of error.

- `HumidifierControl(int humidifierPin, float targetHumidity)`
- `void begin()`
- `bool update()`
- `void setTargetHumidity(float newHumidityTarget)`
- `float getTargetHumidity() const`
- `float getCurrentHumidity() const`

21.5 LightingControl

Regulates lighting based on a target lux via PWM.

- `LightingControl(int sensorPin, int dimmerPin, int targetLux)`
- `void begin()`
- `bool update()`
- `void setTargetLux(int newLuxTarget)`
- `int getTargetLux() const`
- `float getCurrentLux() const`

21.6 DisplayPanel

Manages the LCD UI with rotating pages and error messages.

- `DisplayPanel(int rs, int en, int d4, int d5, int d6, int d7, int maxPeople)`
- `void begin()`
- `void update(int currentPeople, Thermostat&, HumidifierControl&, LightingControl&)`

21.7 Utility functions

Reusable primitives for I/O and sensor readings.

- `bool led(int pin, bool state)`
- `bool ledDimming(int pin, int pwm)`
- `float readTemperatureCelsius(int sensorPin)`
- `float readHumidity()`
- `float readLux(int light_sensor_pin)`
- `bool antiSufferingServo(int angle, Servo& s)`

Chapter 22— Software and dependencies

22.1 Tools

- Arduino IDE or arduino-cli for compilation and upload.
- CMake >= 3.16 for tests.
- C++17 compiler for PC tests.

Using arduino-cli enables automation and integration with test pipelines.

22.2 Libraries

- Servo (Arduino)
- LiquidCrystal (Arduino)
- DHT (for DHT22)
- GoogleTest (tests only)

Arduino libraries are used via standard headers, while GoogleTest is downloaded dynamically during CMake configuration.

22.3 Repository

The project is organized as:

- `Pinacoteca.ino` for firmware;
- `lib/` for modules;
- `test/` for unit tests;
- `diagram.json` for Wokwi simulation.

Chapter 23— Remote Control, App Communication, and Stability Improvements

23.1 Purpose of this chapter

This chapter documents all additions introduced for remote communication with a mobile app, including protocol design, module architecture, validation strategy, software optimizations for memory constraints, and the issues encountered in Wokwi with their corresponding fixes.

23.2 Remote communication architecture

Remote communication is implemented as a layered design:

- **Transport abstraction**: `bluetooth_connection.h` defines a generic interface.
- **Concrete BLE transport**: `ble_link.h` implements the interface for BLE communication.
- **Command processing**: `command_processor.h` parses commands, validates payloads, and updates system state.
- **Gateway orchestration**: `remote_gateway.h` manages serial command input, BLE polling, and periodic state publication.

This separation makes the system maintainable and allows replacing the communication component with minimal code changes.

23.3 Protocol used by the app

The protocol is text-based and intentionally simple for App Inventor integration.

23.3.1 Core commands

- `PING` → `PONG`
- `VER` → `VER:1`
- `GET:STATE`
- `GET:MODE`
- `MANUAL:ON` / `MANUAL:OFF`

23.3.2 Setpoint commands

- `SET:TEMP:<value>` with range $[15.0, 30.0]$
- `SET:HUM:<value>` with range $[40.0, 80.0]$
- `SET:LUX:<value>` with range $[50, 1200]$
- `SET:PEOPLE:<value>` with range $[0, \text{maxPeople}]$

23.3.3 Manual override commands

When in `MANUAL` mode, the app can bypass automatic logic:

- Servo control: `SERVO:OPEN`, `SERVO:CLOSE`, `SERVO:ANGLE:<0..180>`
- Actuator control: `ACT:GREEN:*`, `ACT:RED:*`, `ACT:HEAT:*`, `ACT:COOL:*`, `ACT:HUMIDIFIER:*`
- Lighting PWM: `ACT:PLAFONIERE:PWM:<0..255>`

23.3.4 State payload format

Example:

```
STATE:T=21.8;H=58.0;L=170;P=3;S=90;M=MANUAL;TT=22.5;TH=60.0;TL=180
```

23.3.5 Error model

The protocol reports explicit errors for robust app-side handling:

- `ERR:EMPTY`, `ERR:UNKNOWN`, `ERR:MODE`, `ERR:SERVO`
- Range errors: `ERR:RANGE:*`
- Format errors: `ERR:FORMAT:*`
- Serial overflow guard: `ERR:TOO_LONG`

23.4 Validation and test strategy

The test suite was extended with dedicated remote-control tests:

- protocol correctness (normal commands and invalid commands),
- boundary values for every critical range,
- malformed payload handling,
- AUTO/MANUAL transition behavior,
- fragmented serial input,
- oversized serial command handling,
- disconnected transport behavior,
- queue processing across update cycles.

A fake transport implementation is used in tests to validate logic independently from real BLE hardware.

23.5 Software optimization for stability

Several improvements were introduced to keep execution stable, especially on constrained targets:

- reduced memory footprint by using narrower integer types for pins and counters,
- replaced blocking turnstile behavior with a non-blocking timed servo close,
- reduced dynamic string churn in state payload generation,

- strict numeric parsing to prevent unsafe conversions,
- board-specific build outputs to avoid firmware overwrite between targets,
- separated Wokwi and real-hardware compile workflows.

These changes reduced instability risk and made behavior more deterministic in simulation and deployment.

23.6 Wokwi issues encountered and fixes

During integration, multiple Wokwi-related issues were observed:

23.6.1 Issue A: board and firmware mismatch

If the simulator board and compiled firmware target do not match, simulation appears non-functional.

Fix: use dedicated outputs and commands:

- Wokwi firmware in `build-wokwi/`
- real UNO R4 firmware in `build-r4/`

23.6.2 Issue B: duplicated key in `wokwi.toml`

An accidental duplicated `elf` key caused TOML parsing failure.

Fix: keep a single `elf` entry and validate TOML syntax.

23.6.3 Issue C: servo not visibly moving in simulation

After removing blocking delays, the servo could open/close too quickly in the same control phase.

Fix: introduce non-blocking timed closing and edge-triggered button logic.

23.7 Build workflow (single command)

The project now supports a complete command that runs tests and compiles both targets:

```
make full-test
```

It performs:

1. full unit/integration test run,
2. Wokwi firmware compilation,
3. UNO R4 WiFi firmware compilation.

23.8 No-hardware app testing

A local mock server was added to validate app behavior before hardware deployment:

- script: `tools/mock_arduino_server.py`

- command: `make mock-server`

This enables early testing of UI flows, command sequencing, and parser logic without BLE radio dependency.

23.9 Final deployment note

For final acceptance, BLE communication must still be validated on real UNO R4 WiFi hardware, even if protocol and UI behavior are pre-validated through mocks and Wokwi.